BASICS OF GenAI

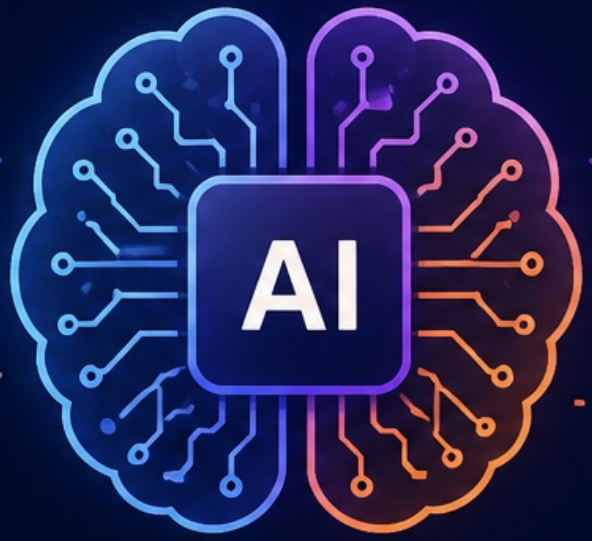
Understand. Create. Innovate.



TEXT GENERATION



IMAGE GENERATION



CODE GENERATION



IDEAS TO IMPACT





UNDERSTAND



CREATE



INNOVATE



EMPOWER

Lecture: RAG

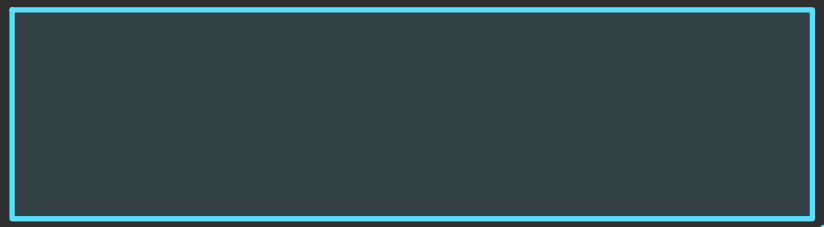
↳ Retrieval Augmented Generation

Agenda

1. Understanding RAG
2. Chucking Techniques
3. Vector Database: Storage and Retrieval
4. Scaling to Production

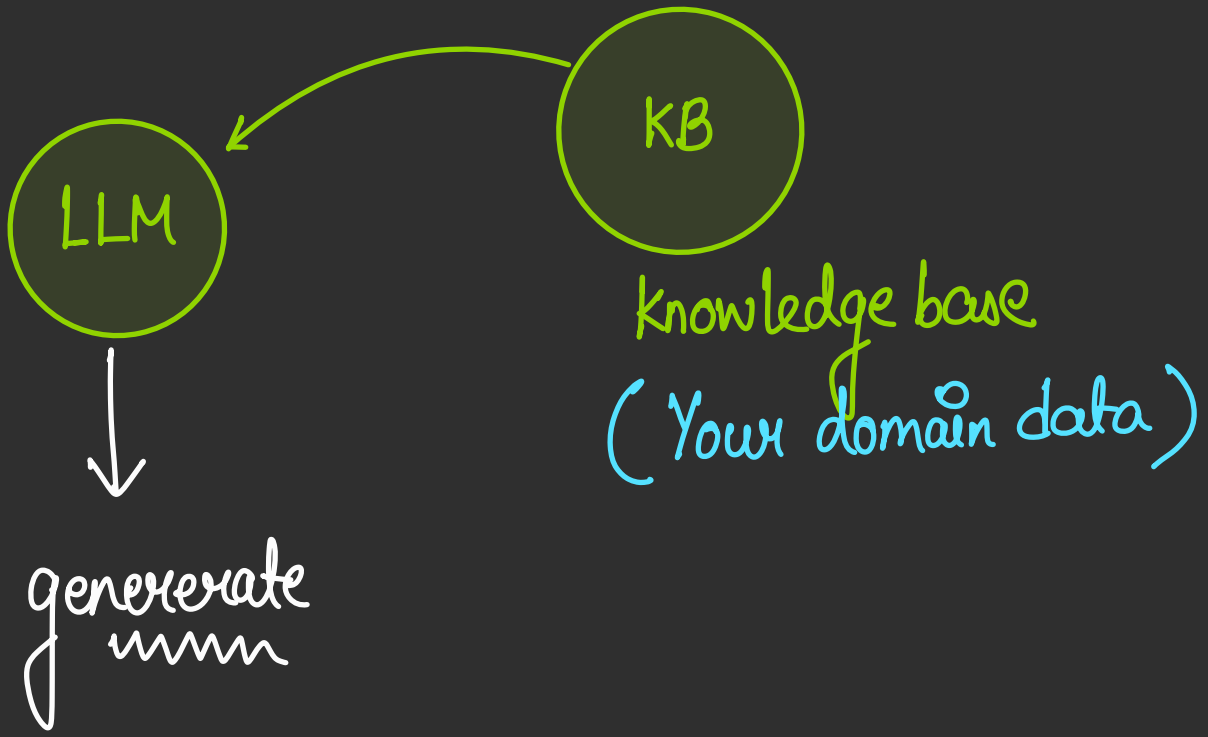
Why RAG?

- ✓ LLM → difficult to train, hence knowledge cutoff
- ✓ LLM → Not specific to any domain, hence hallucination
- ✓ LLM → public data but your domain data (sensitive content)
- ✓ LLM → Context window constraint, RAG solves this
  - → 1m / 128k token



What is RAG?

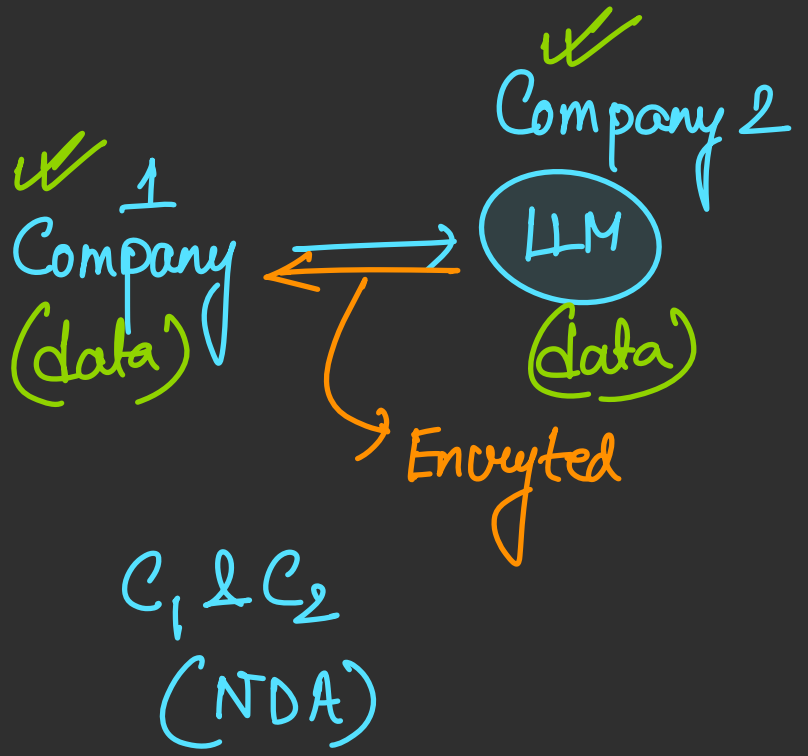
↳ framework (Lewis et al 2020)



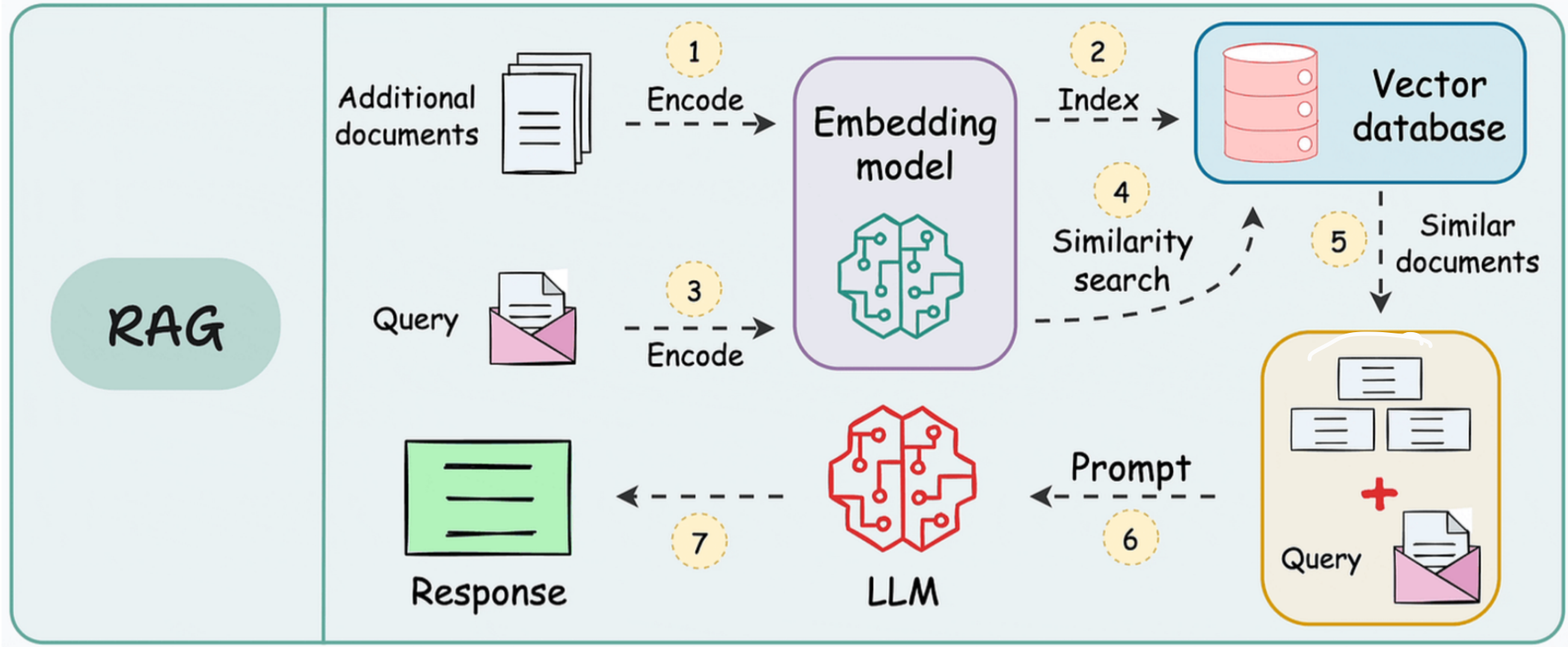
Where to use RAG?

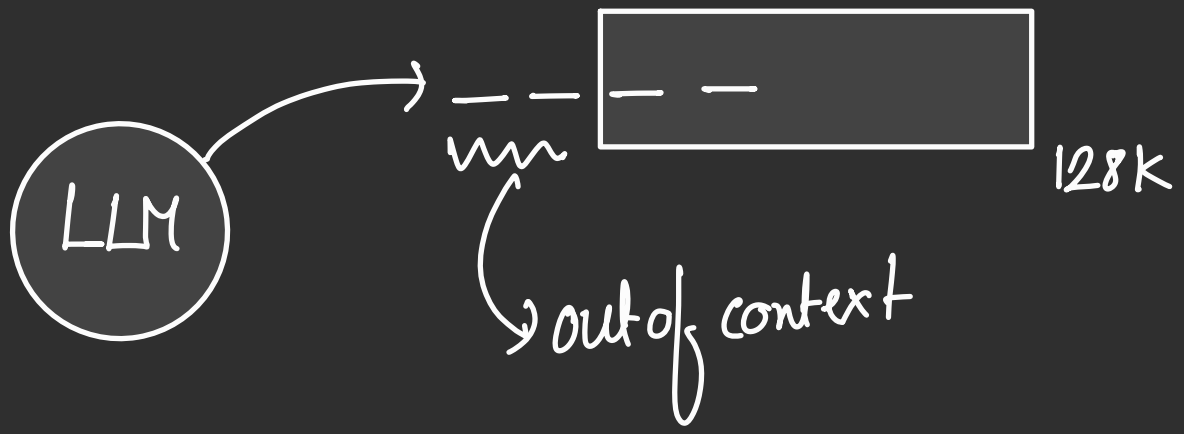
1) Sensitive data

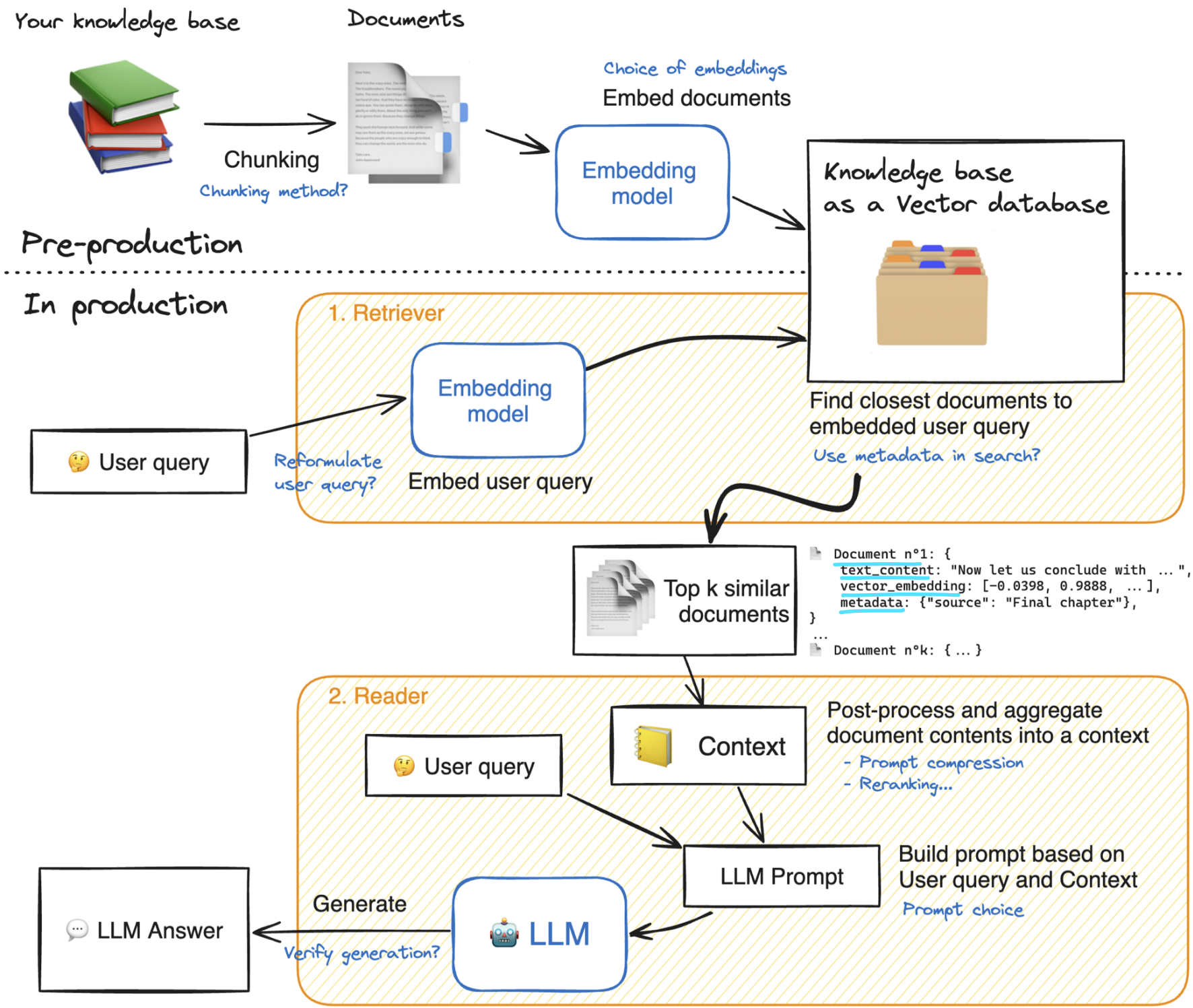
2) When your data states keeps on changing

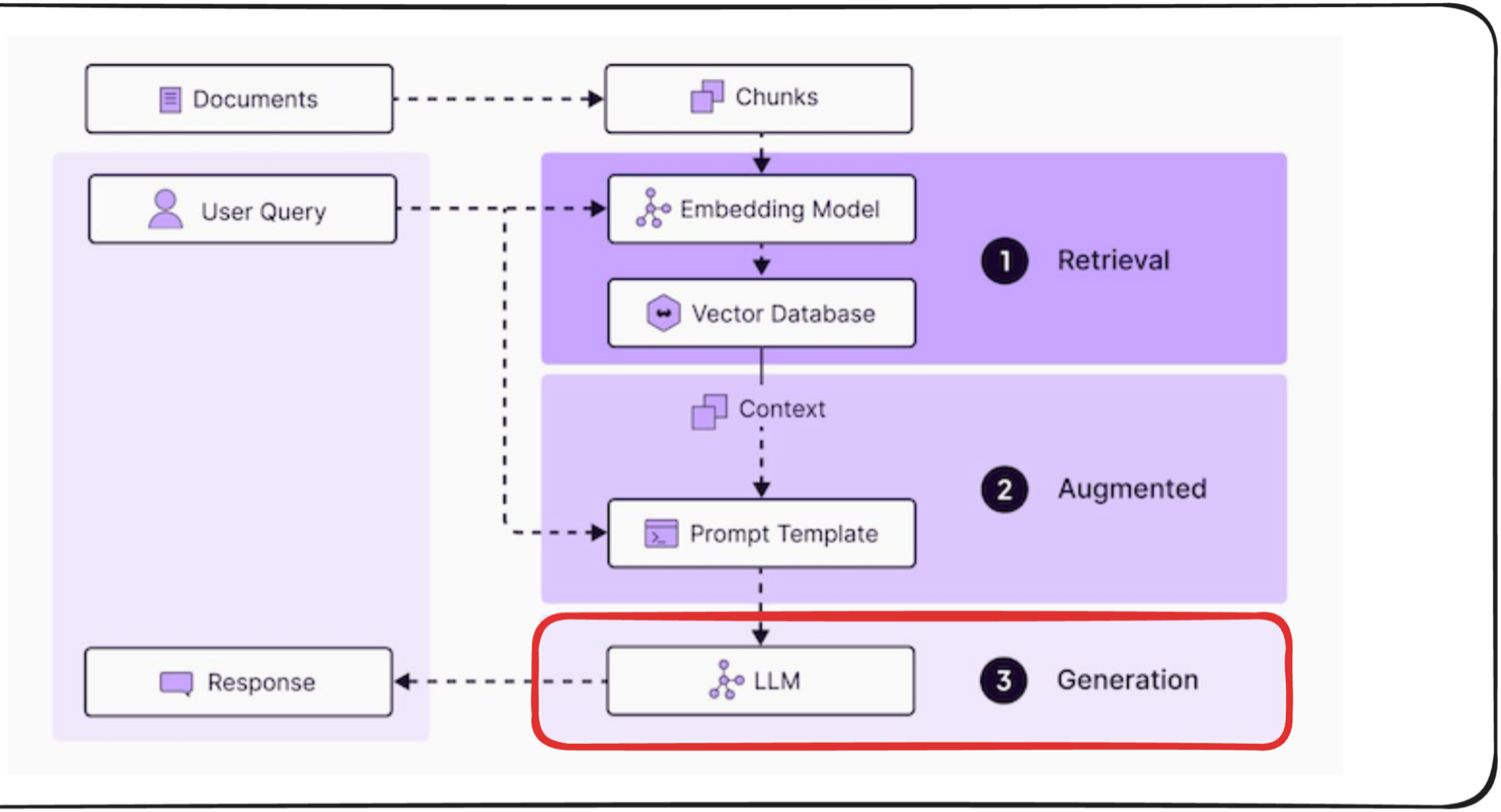


C_1 & C_2
(NDA)


RAG
Additional documents
1
Encode
Embedding model
2
Index
Vector database
4
Similarity search
5
Similar documents
Query
3
Encode
Prompt
6
Query
7
Response
LLM


LLM
128k
out of context


Your knowledge base
Documents
Chunking
Chunking method?
Choice of embeddings
Embed documents
Embedding model
Knowledge base as a Vector database
Pre-production
In production
1. Retriever
Embedding model
User query
Reformulate user query?
Embed user query
Find closest documents to embedded user query
Use metadata in search?
Top k similar documents
Document n°1: {
text_content: "Now let us conclude with ...",
vector_embedding: [-0.0398, 0.9888, ...],
metadata: {"source": "Final chapter"},
}
...
Document n°k: {...}
2. Reader
User query
Context
Post-process and aggregate document contents into a context
- Prompt compression
- Reranking...
LLM Prompt
Build prompt based on User query and Context
Prompt choice
LLM
Generate
Verify generation?
LLM Answer


Documents
Chunks
User Query
Embedding Model
Vector Database
1
Retrieval
Context
Prompt Template
2
Augmented
Response
LLM
3
Generation

Why need Chunking?

LLM ⟶ limited context window

↳ cannot proces all your documents in one go.

"Break our document into managable parts"

"Chunking"

✓ Chunks are too large → Subtopics could get lost

✓ Chunks are too small → Can't see the big pictures

#Chunking strategy

1) Fixed size chunking
2) Token Chunking with overlap
3) Semantic Chunking
4) Recursive Chunking
5) Late Chunking
6) LLM Based Agentic Chunking

Fixed size

D = 3001 token

chunk size = 30

~~100~~ 100 Chunks

101 Chunks

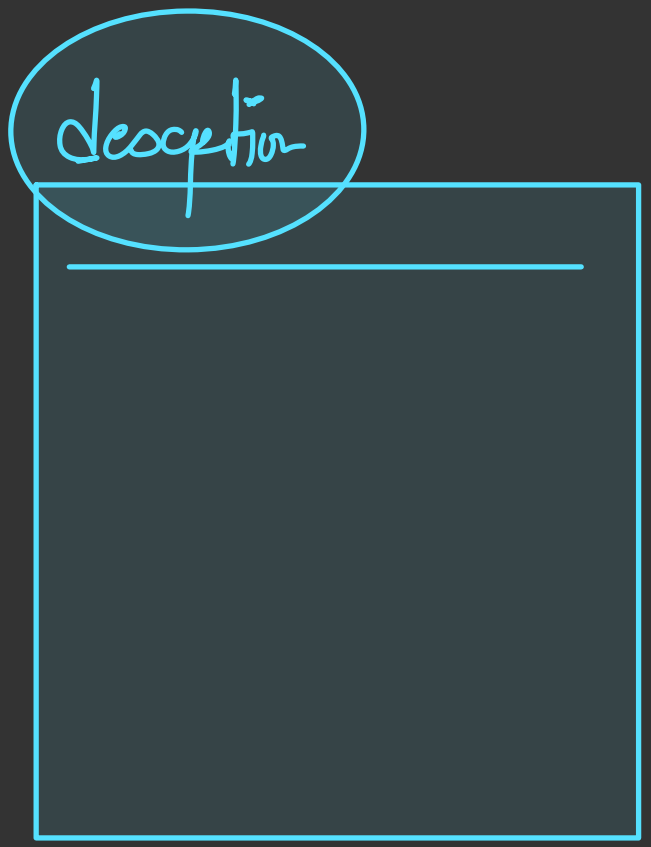
MD

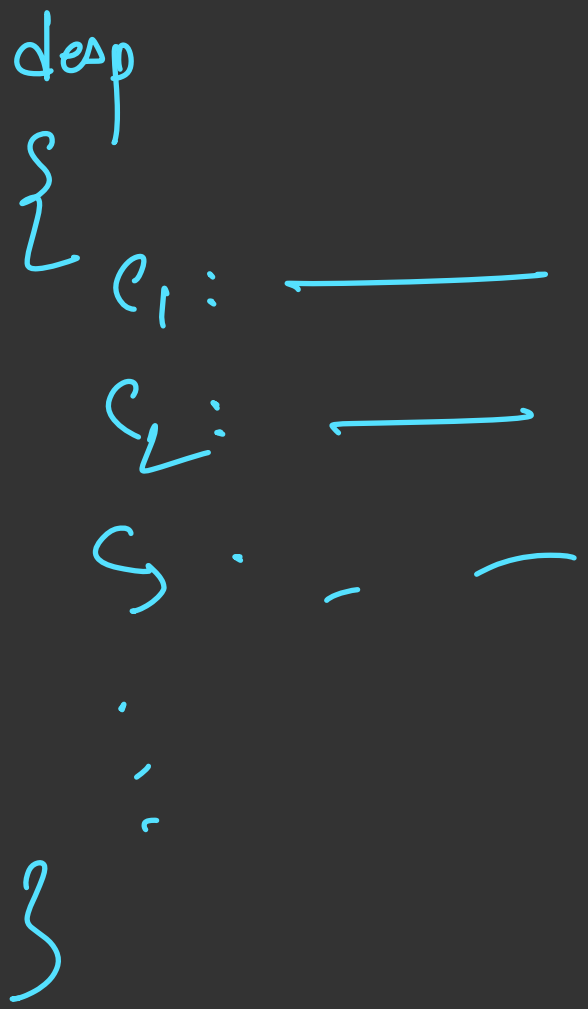
Heading #

Sub Heading ##

dept

Loochuck




deap
{
c_1 :
c_2 :
c_3 :
}